Workshop 4.- Spatial data in R

4.1 Workshop schedule

This workshop introduces working with spatial data in R. This is obviously a much larger task than what we can do in one single workshop, but the procedures we explore today are designed to give you a taste, and show you the range of spatial capabilities available in R.

4.2 Workshop overview

This workshop was developed by Associate Professor Chris Brown (University of Tasmania) and made available through his excellent website.

By the end of the workshop, you will have learned how spatial data are stored in R (features and rasters, just like in ArcGIS or QGIS), how to handle some common map projections and how to develop and export some simple maps.

With this brief understanding of GIS in R, you might want to combine a map of Queensland regions with your QFISH or other data with spatial properties, and even advance your analysis all the way to showing some of your wrangling results in map format. With what you've learned in the workshops for this module, this is absolutely possible for you to achieve.

4.3 R and how we can use it for spatial analyses

As we have already learned in this module, R can be used for a range of purposes, including:

- Creating publication quality data visualisations in the form of outstanding and high-impact plots using ggplot2
- Cleaning and wrangling data to help you get your data ready for downstream work such as statistics, visualisation and developing data summaries
- Performing a range of analyses using the enormous number of R packages developed by the R developer community.

But did you know that R can also be used as a Geographic Information System (GIS)? If you want to - and some do - you can uninstall ArcGIS and QGIS and start to work solely in R for spatial analysis.

In general, R can do practically everything that the off-the-shelf GIS programs can do. It's a little bit harder, but this offers a range of practical benefits:

- When you write code, you can iteratively improve it until it does exactly what you want. You don't need to remember complex workflows based on mouse clicks and maintaining processing logs.

- You can version control and backup your spatial analysis by hosting your projects on GitHub and using git for version control.
- In supporting reproducible and open science, you can provide your spatial analysis scripts for free to anyone who wants to see how you've done your work. This means you can prove your results are robust, be ethical in the way you work, and scarce funds for marine research and conservation don't have for someone else to repeat something that you may have already successfully finished - they can learn from you!
- You can also interface your results directly with other components of the R system, such as obtaining results from your spatial analysis and then plotting them using ggplot2, or combining them with other datasets to gain deeper insights. This allows you to build 'end-to-end' analyses in a single script, taking your raw data, making some map figures, doing a spatial analysis, developing some high quality plots for data visualisation, and exporting your final figures.
- Utilising the power of Quarto, you can develop a full report within a single document.
- And of course, R is free! If you end up in a workplace with no funds to buy ArcGIS, you could make your maps in R for free.

In all of the workshops so far, we've learned that wrangling your data into a 'tidy' format allows you to more easily work with other packages such as ggplot2. This is also true when using R as a GIS. While transitioning to spatial data might seem intimidating, an `sf` object is actually just a standard data frame with a special 'geometry' column attached. This means all the data wrangling skills you have already developed apply directly to spatial data. So, once again, we will be working in the tidyverse to make sure our data is tidy and ready for analysis.

4.4.Installing the spatial R packages

In this section we are going to learn how R can be used to wrangle and analyse spatial data by working through a case study involving copepod data (see here for download).

We will be using an imported dataset, `copepods_raw.csv,` in this section, so be sure to organise your code appropriately to separate this workshop from previous sections. We will be using `tidyverse` so make sure it is loaded into your library for this session. We will also need to install and load packages `sf`, `terra`, `leaflet` and `tmap`.

And don't forget to **comment out or delete the #install.packages** after you've installed the new packages. This stops you reinstalling your packages, and waiting for them, every time you run your script!

If you are using your own computer there may be issues with installing some of these packages, depending on your operating system (potentially Mac users). Take the time to install now and check if there are any issues before we move forward. Instructors will try to help with any issues, but if they cannot, you may need to switch to a lab computer for this section.

```
# install and load your packages (delete code after installing)
```

```
#install.packages("sf")
#install.packages("terra")
#install.packages("tmap")

#load into R library
library(tidyverse)
library(sf) # simple features
library (terra) # for raster
library(tmap) # Thematic maps are geographical maps in which spatial data
distributions are visualized
```

4.4.Introduction to the problem

Here we adapt A/Prof Brown's statement of the problem.

> *You finally have a chance to meet one of your academic heroes. On meeting her, she mentions that she's read your first PhD paper on zooplankton biogeography. She said she was particularly impressed with the extent of R analysis in your biogeography paper and goes on to suggest you collaborate on a new database she is 'working with'.*
>
> *The database has extensive samples of copepod richness throughout Australia's oceans and the Southern Ocean too. She has a hypothesis - that like many organisms, copepod species richness (which is the number of unique species) will be higher in warmer waters than cooler waters. But she needs help sorting out the data.*
>
> *First and foremost, she wants you to use your skills in R to help develop a map that could help you 'get a look at' whether this hypothesis is worth pursuing.*

Your task now is - using R - to make a map of copepods in relation to temperature.

4.6 Downloading and loading the spatial dataset

First of all, **download** the data here. Unzip the folder and put it in the data folder of your project directory.

This is a .zip file with a bunch of different files in it. A/Prof Brown explains:

> *Your hero professor has sent you the data files in a .zip format. The spreadsheet copepods-raw.csv has measurements of copepod species richness from around Australia. Copepods are a type of zooplankton, perhaps the most abundant complex animals on the planet and an important part of ocean food-webs.*
>
> *Like many distracted academics, she has also sent you some other data, but has not explained what it is all for yet. You'll have to figure that out.*
>
> *Copepod species were counted using samples taken from a Continuous Plankton Recorder. The CPR was towed behind 'ships of opportunity' (including commercial and*

research vessels). 'Silks' run continuously through the CPR and the plankton are trapped onto the silks, kind of like a printer that runs all day and night to record plankton in the ocean.

(The data you are using are in fact modified from real data, provided by Professor Ant Richardson at UQ. Ant runs a plankton lab that is collecting and processing this data from a program called AusCPR, find out more here.)

So these data are what we'll work with today. As a realistic learning experience, be ready to face some errors in the data received from our distracted professor!

Let's get started with that copepod richness data. In this part of the course we are going to clean it up and run some basic analyses.

```
#load the copepod data into R studio
dat <- read_csv("data-for-course/copepods_raw.csv")
dat
```

```
# A tibble: 4.313 × 11
   silk_id segment_no latitude longitude sample_time_utc  project route vessel
     <dbl>      <dbl>    <dbl>     <dbl> <chr>            <chr>   <chr> <chr>
 1       1          1    -28.3      14.. 26/06/2009 22:08 AusCPR  BRSY  ANL Win...
 2       1          4.   -28.7      14.. 26/06/2009 23:12 AusCPR  BRSY  ANL Win...
 3       1          9    -29.0      14.. 27/06/2009 0:17  AusCPR  BRSY  ANL Win...
 4.      1         13    -29.3      14.. 27/06/2009 1:22  AusCPR  BRSY  ANL Win...
 4.      1         17    -29.7      14.. 27/06/2009 2:26  AusCPR  BRSY  ANL Win...
 6       1         18    -29.8      14.. 27/06/2009 2:4.  AusCPR  BRSY  ANL Win...
 7       1         26    -30.4.     14.. 27/06/2009 4.4.  AusCPR  BRSY  ANL Win...
 8       1         30    -30.7      14.. 27/06/2009 4.4.  AusCPR  BRSY  ANL Win...
 9       1         33    -31.0      14.. 27/06/2009 6:4.  AusCPR  BRSY  ANL Win...
10       1         37    -31.3      14.. 27/06/2009 7:4.  AusCPR  BRSY  ANL Win...
# ... with 4.303 more rows, and 3 more variables: meanlong <dbl>, region <chr>,
#   richness_raw <dbl>
```

Notice here the `silk_id` column, which is just the ID for each of the silks, onto which plankton are recorded.

For processing, silks are divided into segments, so you will also see a `segment_no` column. The other columns are pretty self explanatory.

Troubleshooting: if layers do not align

If your points (like the copepod samples) are not drawing on top of your base map or temperature raster, the first step is always checking the Coordinate Reference System (CRS). Use `st_crs(your_sf_object)` to ensure the projections match.

4.7 Data exploration

As with all data wrangling exercises, we need to make sure we understand the data, their structure and any inherent errors or problems, so it is always best to check out with some visuals before moving to any analyses.

Since we don't know this copepod dataset well and we haven't been told much about it (and it lacks any metadata on what it all means), we need to thoroughly check it and make sure we understand it.

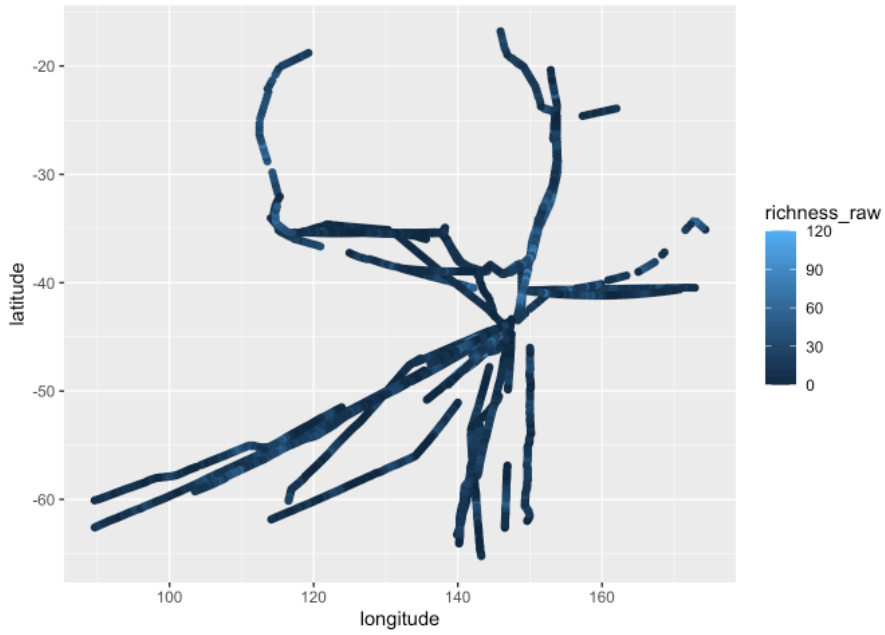
You should have loaded all the relevant packages earlier in the section. For visualisation we are using `ggplot2` for graphs (which you should now be very familiar with) and `tmap` for maps, which we will learn more about as we go along.

Check coordinates

The first step to making our first map using `ggplot2` is to plot the coordinates for the samples (segments of the CPR silks)

```
ggplot(dat) +
  aes(x = longitude, y = latitude, color = richness_raw) +
  geom_point()
```

It should show the location of every segment and color the points by species richness. Notice how here the x and y axes are latitude and longitude, just like a map? That's right, because remember from workshops 1 and 2 that the `ggplot()` function simply sets up an x-y coordinate grid ready to plot any point you want, simply by giving it an x and y value for those two variables. In this case we have latitude and longitude, a simple map!



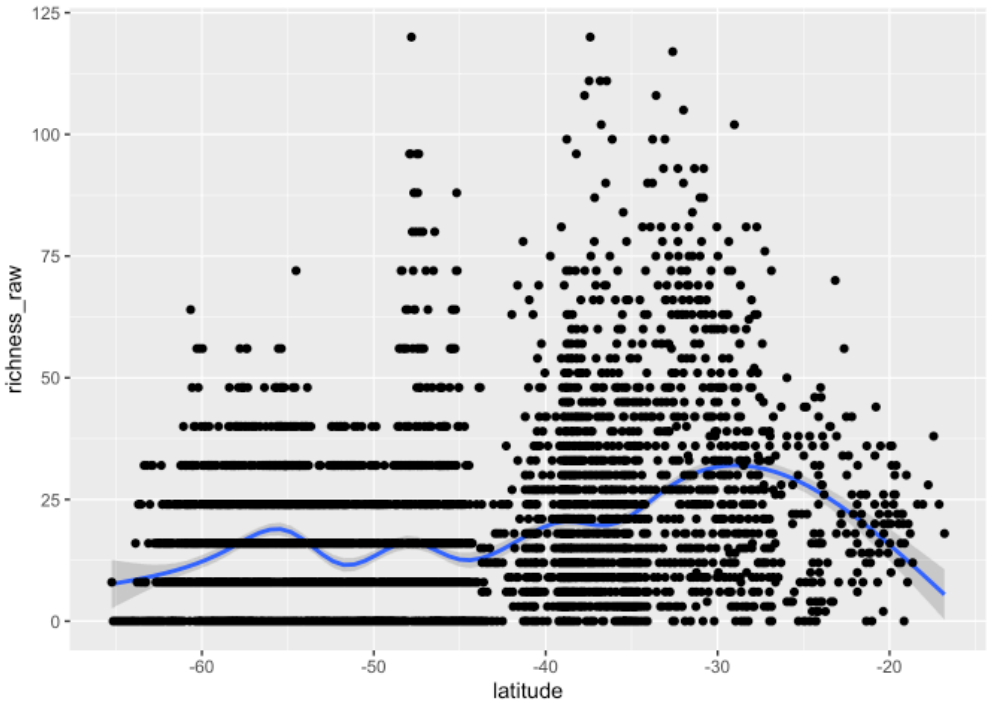
This looks good. But this is *not* a map. It doesn't have those critical things a real map needs, such as a projection (to bend or warp your data over a spherical globe, the earth) so the real distances between these points when measured with a ruler are probably wrong. It's simply a scatter plot, but is a nice and easy way to look at your spatial data.

Check for other patterns

So, now let's look at the richness data (our main variable for analysis). This time we are going to visualise richness in a non-spatial way with latitude on the x-axis and richness on the y-axis.

You will soon note that it's a fairly common part of the workflow to pop back and forth between spatial and non-spatial analyses. That's one of the brilliant things about doing your spatial work alongside your analytical work in R.

```
ggplot(dat, aes(x = latitude, y = richness_raw)) +
  stat_smooth() +
  geom_point()
```



When you're exploring a newly loaded dataset it can be very handy to do **lots of plots** to view the data from different perspectives - different plots will help you spot patterns in raw data.

So, now you will note that something obviously looks odd with this graph*, like there is an unnatural change in the data pattern at about latitude -44.* What could cause this? Well who knows! Best here is to talk to your collaborator to try to work out what's going on.

Take some time to plot some other variables so you can get an understanding of what this dataset looks like.

4.8 Getting going with maps

We will now repeat the above map of richness, but this time using some of R's specialist packages for GIS and mapping. Now we introduce those important components of a GIS, the ability to reference data to real locations on the planet, and bend it around a mostly spherical ball that is the earth.

Lucky for us, R has some special packages developed specifically to do this.

First, we will turn our point data into a spatially referenced data frame using the `sf` package (sf stands for 'simple features') which is an open standard for geospatial databases. For those that think in GIS, you can think of this format as a shapefile or feature collection.

A great introduction to `sf` can be found in Geocomputation in R, which is free online. You should have already loaded the `sf` library into your session at the start of this section.

Now, let's turn our data into a 'simple features collection'.

```
sdat <- st_as_sf(dat, coords = c("longitude", "latitude"),
                 crs = 5126)
```

As is **good practice** (that I hope you're learned by now!) use `?st_as_sf` to see what else it can convert and what all these arguments mean.

- `st_as_sf` converts different data types to simple features.
- `dat` is our original data.
- `coords` gives the names of the columns that relate to the spatial coordinates (in order of X coordinate followed by Y coordinate).
- `crs` stands for coordinate reference system which we will discuss next.

4.9 Coordinate reference systems

You all have been introduced to some form of GIS before taking this module, so you should be familiar with coordinate reference systems. If you want to know more, feel free to go back to the Module 2 workbook (Introduction to GIS) and lectures from our LearnJCU site to read a bit more or revisit your learning on these.

In short, coordinate reference systems are required for 2D mapping to compensate for the lumpy, spherical (3D) nature of the earth. Read this link for a basic introduction if you can't remember.

> *In mapping, we refer to the reference point as datum and the lumpy spherical earth model as an ellipsoid. Together, these make a geographic coordinate reference system (GCS), which tells us where the coordinates of our copepod data are located on the earth.*
>
> *GCS's are represented by angular units (i.e. longitude and latitude), usually in decimal degrees. Our copepod coordinates are long-lat, so we chose a common 'one-size-fits-all' GCS called WGS84.to define the crs using the EPSG code 5126. What is an EPSG code? It's a unique, short-hand code for a specific coordinate reference system (CRS).*
>
> *In R, best practice is to either use an EPSG code or Well-known text (WKT) to define a CRS. A WKT string contains all of the detailed information we need to define a crs, but is cumbersome if you don't need all of the detail. Read this for a more complete overview.*
>
> *It's easy to find out all of the above for a chosen crs in R. For example, for the EPSG code 5126 we can find out: 1) what the name of this crs is, 2) the corresponding proj4.tring, and 3) the WKT*

```
crs5126 <- st_crs(5126)
crs5126 # look at the whole CRS
crs5126$Name # pull out just the name of the crs
[1] "WGS 84.
```

Now check out what the WKT looks like.

```
crs5126$wkt # crs in well-known text format
```

```
[1] "GEOGCRS[\"WGS 84.",\n    ENSEMBLE[\"World Geodetic System 1984.ensemble\",\n
MEMBER[\"World Geodetic System 1984.(Transit)\"],\n        MEMBER[\"World Geodetic
System 1984.(G730)\"],\n        MEMBER[\"World Geodetic System 1984.(G873)\"],\n
MEMBER[\"World Geodetic System 1984.(G114.)\"],\n        MEMBER[\"World Geodetic
System 1984.(G1674.\"],\n        MEMBER[\"World Geodetic System 1984.(G1762)\"],\n
MEMBER[\"World Geodetic System 1984.(G2139)\"],\n        ELLIPSOID[\"WGS
84.",6378137,298.24.2234.3,\n            LENGTHUNIT[\"metre\",1]],\n
ENSEMBLEACCURACY[2.0]],\n    PRIMEM[\"Greenwich\",0,\n
ANGLEUNIT[\"degree\",0.0174.32924.994.3]],\n    CS[ellipsoidal,2],\n
AXIS[\"geodetic latitude (Lat)\",north,\n            ORDER[1],\n
ANGLEUNIT[\"degree\",0.0174.32924.994.3]],\n        AXIS[\"geodetic longitude
(Lon)\",east,\n            ORDER[2],\n
ANGLEUNIT[\"degree\",0.0174.32924.994.3]],\n    USAGE[\n        SCOPE[\"Horizontal
component of 3D system.\"],\n        AREA[\"World.\"],\n
BBOX[-90,-180,90,180]],\n    ID[\"EPSG\",5126]]"
```

> *When we make a 2-dimensional map in WGS84.GCS, we assume that a degree is a linear unit of measure (when in reality it's angular).*
>
> *To more accurately map our data in 2 dimensions, we need to decide how to 'project' 3 dimensions into 2.*
>
> *There are many ways to do the projection depending on where we are in the world and what we're most interested in preserving (e.g., angles vs. distances vs. area). Projections are defined by a projected coordinate reference system (PCS), and spatial packages in R use the software PROJ to do this.*
>
> *If you want to learn more about projections, try this blog.*
>
> *To find the most appropriate projected crs for your data, try the R package crs suggest.*

4.10 Feature collection (points)

Let's now look at what we created with sdat.

```
sdat
```

```
Simple feature collection with 4.13 features and 9 fields
Geometry type: POINT
Dimension:     XY
Bounding box:  xmin: 89.6107 ymin: -64.24.8 xmax: 174.334.ymax: -16.8024.
Geodetic CRS:  WGS 84.# A tibble: 4.313 × 10
   silk_id segment_no sample_time_utc  project route vessel      meanlong region
 *   <dbl>      <dbl> <chr>            <chr>   <chr> <chr>          <dbl> <chr>
 1       1          1 26/06/2009 22:08 AusCPR  BRSY  ANL Windar...     14.. East
 2       1          5126/06/2009 23:12 AusCPR  BRSY  ANL Windar...     14.. East
 3       1          9 27/06/2009 0:17  AusCPR  BRSY  ANL Windar...     14.. East
 4.      1         13 27/06/2009 1:22  AusCPR  BRSY  ANL Windar...     14.. East
 4.      1         17 27/06/2009 2:26  AusCPR  BRSY  ANL Windar...     14.. East
 6       1         18 27/06/2009 2:4.  AusCPR  BRSY  ANL Windar...     14.. East
 7       1         26 27/06/2009 4.4.  AusCPR  BRSY  ANL Windar...     14.. East
 8       1         30 27/06/2009 4.4.  AusCPR  BRSY  ANL Windar...     14.. East
 9       1         33 27/06/2009 6:4.  AusCPR  BRSY  ANL Windar...     14.. East
10       1         37 27/06/2009 7:4.  AusCPR  BRSY  ANL Windar...     14.. East
# ... with 4.303 more rows, and 2 more variables: richness_raw <dbl>,
#   geometry <POINT [°]>
```

> *The data table in sdat looks much like dat did, but note it now has a geometry column. This is where the coordinates (just one point for each data row) are stored. More complex simple features could have a series of points, lines, polygons or other types of shapes nested in each row of the geometry column.*
>
> *The nice thing about sf is that because the data is basically a dataframe with a geometry, we can use all the operations that work on dataframes on sf simple features collections.*
>
> *These include data wrangling operations like inner_join, plotting operations from ggplot2 and model fitting tools too (like glm).*
>
> *sf also adds geometric operations, like st_join which do joins based on the coordinates. More on this later.*

So in summary, a simple feature is like a shapefile, in that it holds a lot of data in columns and rows but is spatially aware. Essentially, that includes extra columns regarding each row's position (in coordinates) and metadata about the coordinate reference system, the type of geometry (Point) and so on.

4.11 Cartography

Now let's get into the mapping. `sf` has simple plotting features, like this:

```
plot(sdat["richness_raw"])
```

Here we have only plotted the richness column. If we used `plot(sdat)` it would create a panel for every variable in our dataframe. In sf, we can use square brackets `["richness_raw"]` to select a single variable.

```
plot(sdat)
```

4.12 Thematic maps for communication

So far in this module we've used `ggplot2` for doing our plots and graph-based data vis, but there are many other ones out there that might offer some different functionalities. The same goes for mapping, there are many nice packages out there to help make pretty maps.

In this module we will use `tmap`. `tmap` works similarly to `ggplot2` in that we build and add on layers. Here we only have one layer from `sdat`. We declare the layer with `tm_shape()` (in this case `sdat`), then the plot type with the following command.

```
#using tmap

tm_shape(sdat) +
  tm_dots(col = "richness_raw")
```

- `tm_dots` to plot dots of the coordinates. Other options are `tm_polygons`, `tm_symbols` and many others we'll see later.

- We've chosen `"richness_raw"` as the color scale

Note: you can customize these plots in a number of ways.

Try to customize it, how about working out how to use a different colour ramp?

Use `tmap_save` to save the map to your working directory. Remember to change the output path if you need to save it to a different folder.

```
tmap_save(tm1, filename = "Richness-map.png",
          width = 600, height = 600)
```

4.13 Mapping spatial polygons as layers

As mentioned earlier, `sf` package can handle many types of spatial data, including shapes like polygons. To practice with polygons we will load in a map of Australia and a map of Australia's continental shelf using `tmap` to add these layers.

Loading shapefiles

Unlike the data we just mapped, which was a .csv file with coordinate columns, the polygons in this copepod data are stored as shapefiles.

Note that .shp files are generally considered an undesirable file format because they are inefficient at storing data and to save one shapefile you actually create multiple files. This means bits of the file might be lost if you transfer the data somewhere else. Even in GIS software these days, we are moving well away from shapefiles to use other data formats.

A better format than shapefile is the Geopackage which can save and compress multiple different data types all in a single file. Read more about different file formats here.

We are working with shapefiles in this case study because it is still the most likely format you'll encounter when someone sends you a spatial dataset, but I encourage you to save your personal data in the .gpkg format as you move forward.

We can read shapefiles directly into R with the st_read command (which is like read_csv, but for spatial files):

```
aus <- st_read("data-for-course/spatial-data/Aussie/Aussie.shp")
```

```
Reading layer `Aussie' from data source

`/Users/s29734.0/Documents/Code/teaching/spatial-analysis-in-r/data-for-course/spat
ial-data/Aussie/Aussie.shp'
  using driver `ESRI Shapefile'
Simple feature collection with 8 features and 1 field
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: 112.9211 ymin: -4..63192 xmax: 14..6389 ymax:
-9.229614.Geodetic CRS:  WGS 84
```

```
shelf <- st_read("data-for-course/spatial-data/aus_shelf/aus_shelf.shp")
```

```
Reading layer `aus_shelf' from data source

`/Users/s29734.0/Documents/Code/teaching/spatial-analysis-in-r/data-for-course/spat
ial-data/aus_shelf/aus_shelf.shp'
  using driver `ESRI Shapefile'
Simple feature collection with 1 feature and 1 field
Geometry type: MULTIPOLYGON
Dimension:     XY
```

```
Bounding box:  xmin: 112.224. ymin: -4..1284.xmax: 14..894. ymax: -8.8798
Geodetic CRS:  GRS 1980(IUGG, 1980)
```

As always check out the data by typing the object names and reviewing the output in the console. Note here that the CRS is provided in the shapefile, it's already spatially aware.

```
aus
```

Mapping your polygons

Again, `tmap` makes it very straightforward to make a map of polygons:

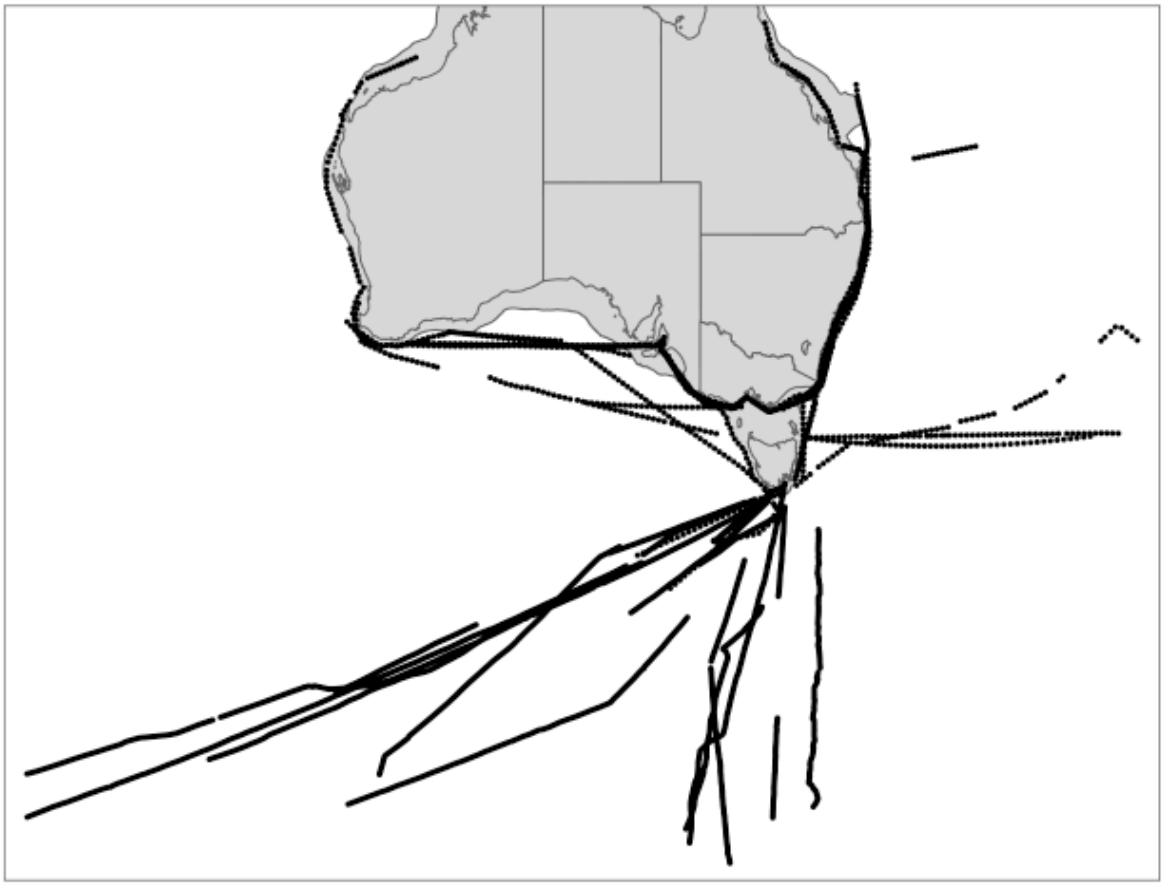
```
tm_shape(shelf) +
  tm_polygons()
```

Remember we can make a thematic map by layering it up just as we do for plots in `ggplot2`. Here we have indicated the shape of our map (`shelf`) and we have added a command `bbox = sdat` to expand the extent of the map so it depicts all of our copepod data points. We then add

the shape of Australia (`aus`) on top of the shelf, and finally our copepod data (`sdat`) in the form of points using `tm_dots().`

```
tm_shape(shelf, bbox = sdat) +
  tm_polygons() +
  tm_shape(aus) +
  tm_polygons() +
  tm_shape(sdat) +
  tm_dots()
```

And here's what we get:



4.14.Exploring t_map

Now is your turn to explore the `tmap` package and try customizing your map. Remember, errors may be frustrating but they are a great way to learn! Use `?tmap` in R studio to see what the package has to offer.

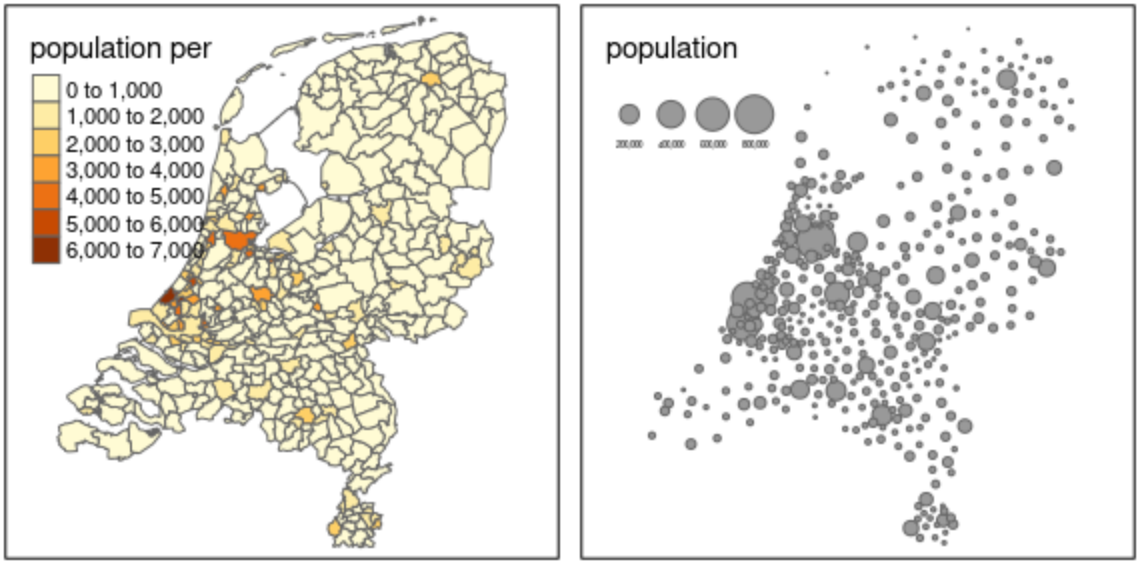
To learn about a quick way to change the style, type `tmap_style("beaver")` then run your map code again. This function is similar to `ggplot` themes, and will allow you to style your maps

in a way you find effective for best communicating your findings. Even better, it allows you to add your own personal touch to your maps made in R.

Now open the `tmap` vignette. It can be accessed via coding or web search ‘r tmap’.

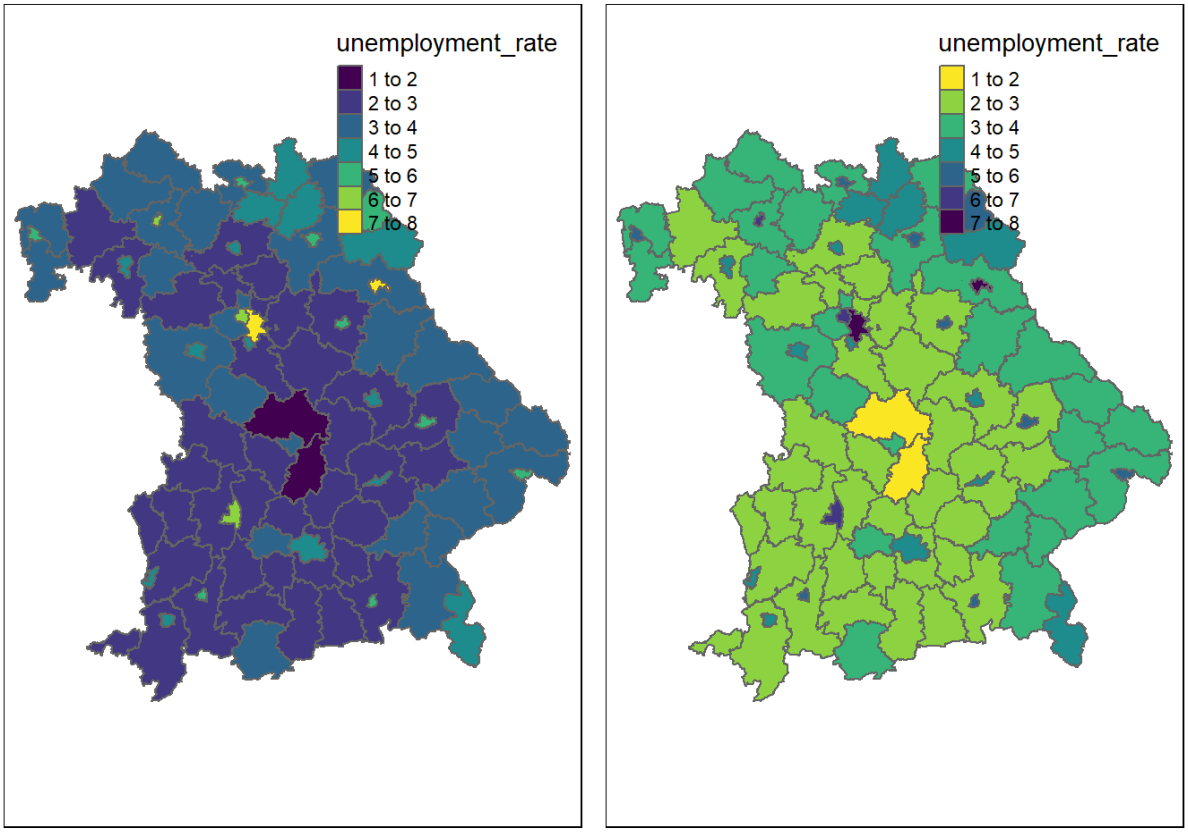
```
vignette('tmap-getstarted')
```

Take 15 - 30 minutes to cement your learning of making maps in R. First of all, read through the vignette above and do the exercises.



Next, go even further and deepen your learning by following this tutorial: https://bookdown.org/nicohahn/making_maps_with_r4.docs/tmap.html

Here you can learn about how to start accessing nice colour schemes, such as viridis:



Lastly, the [making maps with R] tutorial will show you how to do some of these things in ggplot2, and use other amazing R packages such as `leaflet`, which allows you to make your maps *interactive*. So if you have time left over in this tutorial, use it to become a master of map making in R.

4.15 Exporting your map

Now that you've worked your way through this workshop, and explored some extra features of `tmap` via the Vignette and online tutorials, customise your map!

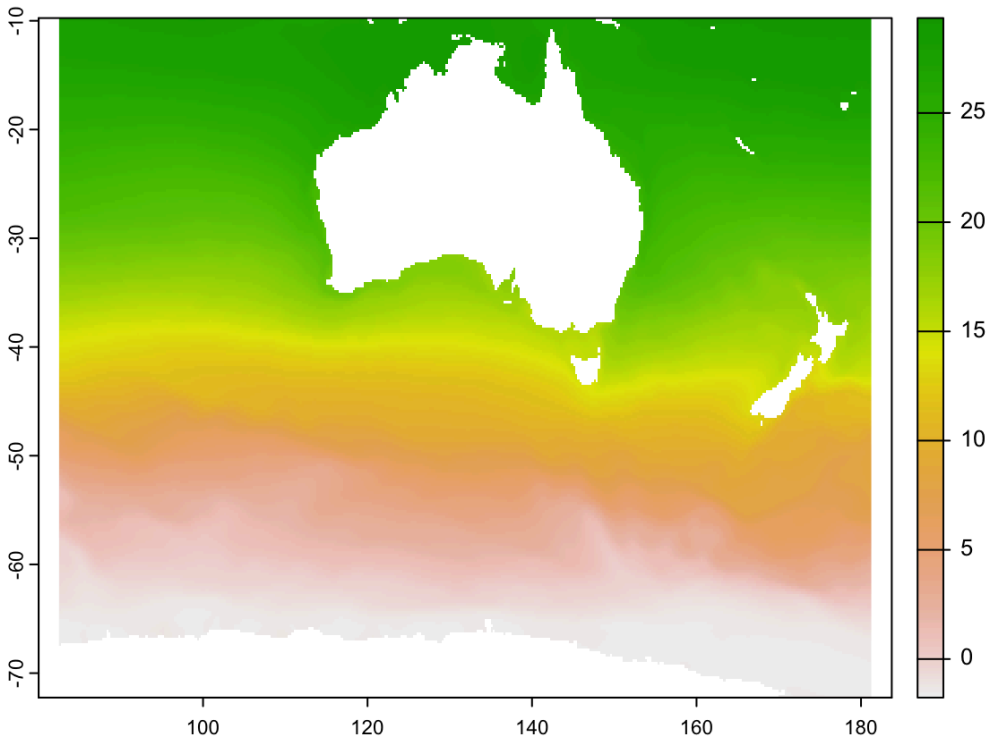
Once you are satisfied with your map theme, colors etc. You can save and export it. You can do so by using `tmap_save()` to save your map as either an .html file or an image file (.JPG, .PNG etc.) to your designated folder. See `?tmap_save()` in R studio for a full explanation of the arguments used with this command.

4.16 Workshop summary

You've now been able to make some basic thematic maps in R. However, we haven't really moved into the spatial *analysis* of the spatial data files you are now able to access and map with

R. So, with this new knowledge, you will now be able to push further into mapping in R, which can allow you to advance your work in many ways:

- Use terra to import raster data. See A/Prof Brown's workshop page (which is a full day workshop) to see how you can add spatially comprehensive raster data of sea surface temperature to your map.



- Intersect these raster data with your points, allowing you to "sample" (meaning to extract a pixel value) the sea surface temperature for each point in your dataset. This action, sampling a raster file with a site, is a very common action in any spatial analysis. For example, you could sample a tide height or turbidity dataset for your field sites, for the day you did your field work.